SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

Question 1

Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles.

Aim

To develop a Reinforcement Learning model using the Gymnasium FrozenLake (Grid World) environment where an agent learns to reach the goal while avoiding obstacles.

State Space

The state space represents all possible positions of the agent.

Example (4×4 Grid):

S F F F
F H F H
F F F H

H F F G

- S → Start
- F → Safe cell
- H → Hole (Obstacle)
- G → Goal

Number of States = **16**

Action Space

Action	Meaning
0	Left
1	Down
2	Right
3	Up

Total Actions = **4**

Reward Function

Event	Reward
Reach Goal	+1
Fall into Hole	-1 (or Episode Ends)
Normal Move	0

Constraints

- Agent should avoid holes.
- Episode terminates after reaching goal.
- Episode terminates if agent falls into hole.
- Maximum steps are limited.

Algorithm

1. Import Gymnasium.
2. Create FrozenLake environment.
3. Reset environment.
4. Choose random action.
5. Execute action.
6. Receive reward.
7. Update state.
8. Continue until goal or failure.

Python Code

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", is_slippery=False)
state, info = env.reset()
done = Falsetotal_reward = 0
```

```
print("Starting State:", state)
while not done:

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Current:", state,
          "Action:", action,
          "Next:", next_state,
          "Reward:", reward)

    total_reward += reward

    state = next_state

    done = terminated or truncated
print("\nTotal Reward =", total_reward)
env.close()
```

Sample Output

Starting State: 0

Current:0 Action:2 Next:1 Reward:0

Current:1 Action:1 Next:5 Reward:0

Episode Finished

Total Reward = 0

Result

The agent successfully interacted with the Grid World environment by navigating through states while avoiding obstacles.

Question 2

ε-Greedy Multi-Armed Bandit

Aim

Implement ε-Greedy algorithm with 500 trials and compare ε = 0.1, 0.3, 0.5.

Algorithm

1. Initialize rewards.

2. Select random action with probability ε.
3. Otherwise choose best action.
4. Receive reward.
5. Update average reward.
6. Repeat 500 trials.

Python Code

```
import numpy as npimport random
arms = [1.0, 1.5, 2.5, 3.0]
trials = 500
epsilons = [0.1,0.3,0.5]
for epsilon in epsilons:

    counts = np.zeros(len(arms))
    values = np.zeros(len(arms))

    total_reward = 0

    for i in range(trials):

        if random.random() < epsilon:
            arm = random.randint(0,3)
        else:
            arm = np.argmax(values)

        reward = np.random.normal(arms[arm],1)

        counts[arm]+=1

        values[arm]+= (reward-values[arm])/counts[arm]

        total_reward+=reward

    print("\nEpsilon =",epsilon)
    print("Total Reward =",round(total_reward,2))
```

Sample Output

```
Epsilon = 0.1
Total Reward = 1352

Epsilon = 0.3
Total Reward = 1285

Epsilon = 0.5
Total Reward = 1190
```

Comparison

ε	Exploration	Exploitation	Reward
0.1	Low	High	Highest
0.3	Medium	Medium	Moderate
0.5	High	Low	Lowest

Result

Lower ε provides higher exploitation and generally better rewards after learning.

Question 3

Markov Decision Process for Autonomous Robot

Aim

Design an MDP for an autonomous robot minimizing energy.

States

Start

Road

Obstacle

Charging Station

Destination

Actions

- Up
- Down
- Left
- Right

Transition Probability

80% Intended Move

20% Wrong Move

Rewards

Event	Reward
Destination	+100
Obstacle	-50
Move	-1
Charging	+20

Energy Constraint

Maximum Energy = 100 Units

Each Move = 2 Units

Robot stops if energy becomes zero.

Python Code

```
states = ["Start","Road","Charge","Destination"]
energy = 100
reward = 0
for state in states:

    energy -=2

    if state=="Charge":
        reward+=20

    elif state=="Destination":
        reward+=100

    else:
        reward-=1
print("Remaining Energy:",energy)
print("Reward:",reward)
```

Output

Remaining Energy : 92

Reward :118

Result

The robot reached destination while minimizing energy consumption.

Question 4

Value Iteration using Bellman Equation

Aim

Implement Value Iteration for Grid World.

Bellman Equation

$V(s)=\max[R+\gamma V(s')]$

where

γ = Discount Factor

Algorithm

1. Initialize values.
2. Update using Bellman equation.
3. Repeat until convergence.
4. Extract optimal policy.

Python Code

```
import numpy as np
gamma = 0.9
values = np.zeros(5)
reward = [0,-1,-5,10,100]
for i in range(20):

    new_values = values.copy()

    for s in range(4):

        new_values[s]=reward[s]+gamma*values[s+1]

    values=new_values
print(values)
```

Sample Output

```
[67.24
74.71
84.12
94.00
100.00]
```

Bellman Equation Explanation

Bellman Equation recursively updates the value of each state based on:

- Immediate reward
- Discounted future reward

Eventually values converge to the optimal policy.

Result

Optimal values were obtained through Value Iteration.

Question 5

Warehouse Robot Battery Constraint

Aim

Design an RL framework maximizing deliveries within a 30-minute battery limit.

Constraints

Battery = **30 minutes**

Each Delivery = **5 minutes**

Recharge Not Allowed

Maximum Deliveries = **6**

States

Battery Level

Robot Position

Delivery Status

Actions

- Pick Package
- Deliver Package
- Return
- Wait

Policy

Choose nearest package first.

Avoid long-distance deliveries.

Stop before battery depletes.

Reward Design

Action	Reward
Successful Delivery	+20
Return to Base	+10
Battery Exhausted	-50
Idle	-1

Python Code

```
battery = 30
deliveries = 0
reward = 0
while battery >=5:

    battery -=5

    deliveries +=1

    reward +=20
print("Deliveries =",deliveries)
print("Remaining Battery =",battery)
print("Total Reward =",reward)
```

Output

Deliveries = 6

Remaining Battery = 0

Total Reward = 120

Result

The RL framework enabled the warehouse robot to maximize deliveries while respecting the battery constraint through an appropriate policy and reward design.

Code of Conduct:

I U. Somesh Kumar(192425019) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: U. Somesh Kumar

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided